

Lucas Pérez

```

classDiagram
    class Calculable {
        +calcularCosto() double
    }
    class Browsable {
        +mostrarClases() void
    }
    class GarageLentoException {
        +GarageLentoException(specificoMensaje: int, especificoDisponibles: int)
    }
    class Moto {
        +Moto(paterno, marca, modelo, horas)
        +getPaterno() String
        +getMarca() String
        +getModelo() String
        +getHoras() int
        +getPaterno() String
        +getMarca() String
        +getModelo() String
        +getHoras() int
        +getPaterno() String
        +getMarca() String
        +getModelo() String
        +getHoras() int
    }
    class Auto {
        +Auto(paterno, marca, modelo, horas)
        +getPaterno() String
        +getMarca() String
        +getModelo() String
        +getHoras() int
    }
    class Camion {
        +Camion(paterno, marca, modelo, horas)
        +getPaterno() String
        +getMarca() String
        +getModelo() String
        +getHoras() int
    }
    class Garage {
        +capacidadMaxima: int
        +vehiculos: Array<IVehicle>
        +Garage(specificoMensaje: int)
        +mostrarMensaje() void
        +registrarGarage(paterno: String, vehiculo: String, modelo: String) void
        +eliminarGarage(paterno: String, modelo: String) void
        +mostrarCosto() void
        +mostrarDisponibilidad() void
        +getPaterno() String
        +getMarca() String
        +getModelo() String
        +getHoras() int
        +getPaterno() String
        +getMarca() String
        +getModelo() String
        +getHoras() int
    }
    class Validador {
        +validarCamion(paterno: String, modelo: String) void
        +validarAuto(paterno: String, modelo: String) void
        +validarCamion(paterno: String, modelo: String) void
        +validarAuto(paterno: String, modelo: String) void
    }
    class HorarioValidaException {
        +HorarioValidaException(horas: int)
    }
    class Reporte {
        +garage: Garage
        +Reporte(garage: Garage)
        +generarReporte() void
        +getPaterno() String
        +getMarca() String
        +getModelo() String
        +getHoras() int
    }
    class Main {
        +scanner: Scanner
        +garage: Garage
        +reporte: Reporte
        +main() void
    }
    Calculable <|-- Moto
    Calculable <|-- Auto
    Calculable <|-- Camion
    Browsable <|-- Moto
    Browsable <|-- Auto
    Browsable <|-- Camion
    GarageLentoException <|-- Garage
    Garage <|-- Validador
    Validador <|-- HorarioValidaException
    Garage <|-- Reporte
    Main --> Garage
    Main --> Validador
    Main --> Reporte
    
```

```

    usecaseDiagram
        actor Usuario
        usecase U1[Inicializar Garage]
        usecase U2[Registrar Ingreso de Vehiculo]
        usecase U3[Registrar Salida de Vehiculo]
        usecase U4[Listar Vehiculos]
        usecase U5[Ver Estado del Garage]
        usecase U6[Generar Reporte]
        usecase U7[Validar Datos]
        usecase U8[Verificar Espacio Disponible]
        usecase U9[Buscar Vehiculo por Patente]
        usecase U10[Mostrar Resumen de Salida]

        Usuario -- U1
        Usuario -- U2
        Usuario -- U3
        Usuario -- U4
        Usuario -- U5
        Usuario -- U6

        U2 -.-> U7
        U7 -.-> U8
        U8 -.-> U9
        U9 -.-> U10

        U3 -.-> U7
        U7 -.-> U8
        U8 -.-> U9
        U9 -.-> U10
    
```

The diagram illustrates the functional requirements of a Garage System. It features a single actor, the **Usuario** (User), who interacts with six primary use cases: **Inicializar Garage**, **Registrar Ingreso de Vehiculo**, **Registrar Salida de Vehiculo**, **Listar Vehiculos**, **Ver Estado del Garage**, and **Generar Reporte**. The **Registrar Ingreso de Vehiculo** and **Registrar Salida de Vehiculo** use cases are further detailed by a sequence of sub-use cases: **Validar Datos**, **Verificar Espacio Disponible**, **Buscar Vehiculo por Patente**, and **Mostrar Resumen de Salida**. These sub-use cases are connected by dashed arrows, indicating a sequential flow. The diagram is organized into two swimlanes: the actor swimlane on the left and the system swimlane on the right, separated by a vertical dashed line.